Ders 13 Genişletilmiş Notu: İkili Arama Ağacı, Heap Ağacı ve Heap Sort

Ders 13 Genişletilmiş Notu: İkili Arama Ağacı, Heap Ağacı ve Heap Sort

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Ağaçlar ve Dolaşma Yöntemlerinin Tekrarı
- 2. Neden İkili Arama Ağacı?
- 3. Ağacın Yüksekliği ve Denge
- 4. İkili Arama Ağacına Ekleme
- 5. İkili Arama Ağacından Silme
- 6. İkili Ağaç Neden Dizide Değil de Bağlı Yapıda Saklanır?
- 7. İkili Arama Ağacının Kodları
- 8. Ara Bilgi: Bellekteki Heap Bölgesi
- 9. Heap Ağacının Üç Özelliği
- 10. Heap Ağacına Eleman Ekleme
- 11. Heap Ağacından Kökü Silme
- 12. Heap Ağacının Dizide Saklanması
- 13. Heap Sort
- 14. Heap Sort'un Karmaşıklığı
- 15. Heap Sort'un Algoritması ve Küçük Bir Örnek
- 16. Kuyruk Soruları: `isFull`, Dairesel Kuyruk ve Yığınla Kuyruk

1. Ağaçlar ve Dolaşma Yöntemlerinin Tekrarı

1.1 Ağaç Neden Kullanılır?

Hoca derse geçen haftanın konusu olan ağaç veri yapısını hatırlatarak başladı. Ağaç, hiyerarşik durumlar için uygun bir yapıdır. Bir karar ağacı üzerinde ilerlemek gerekebilir. Bir işletim sisteminin dizin (directory) yapısı da aslında bir ağaçtır: En üstte kök (root) dizini vardır, her şey onun altındadır, dizinin altında yine dizinler bulunur. Bir seviyelendirmeyle çözülmesi gereken problemlerde ağaç kullanmak uygundur.

Temelde bir ağaçta kökün altında birden fazla alt ağaç (subtree) bulunur. Önce genel ağaçlar, sonra ikili ağaçlar (binary tree) görülmüştü. Aritmetik ifadelerin çözümünde de ikili ağaçtan faydalanılabileceği gösterilmişti.

Alt ağaç kavramı buradaki bütün algoritmaların temelidir. Sol alt ağaç (left subtree) da kendi içinde bir ağaçtır: Onun da bir kökü, solu ve sağı vardır. Sağ alt ağaç (right subtree) için de aynısı geçerlidir.

1.2 Üç Dolaşma Yöntemi

Hiyerarşik bir yapıda nasıl ilerleneceği sorusunun üç temel cevabı vardır. Hangisinin kullanılacağına probleme göre karar verilir. Hoca bunları `a + b` ifadesi üzerinden hatırlattı:

- **Preorder:** Önce kök, sonra sol alt ağaç, sonra sağ alt ağaç. İfade `+ a b` olarak okunur. Bu, ayrık matematikte görülen önek (prefix) gösterimdir.
- **Inorder:** Önce sol alt ağaç, sonra kök, sonra sağ alt ağaç. İfade `a + b` olarak okunur.
- **Postorder:** Önce sol alt ağaç, sonra sağ alt ağaç, sonra kök. İfade `a b +` olarak okunur. Bu da sonek (postfix) gösterimdir.

Preorder'da kök yazıldıktan sonra sol alt ağaçta ne varsa önce o halledilir. Sağ tarafa ancak sol bittikten sonra geçilir. Her kök için önce sol, sonra sağ bitirilecekse, her alt ağaçta da aynı iş tekrar eder. Bu yüzden çözüm doğal olarak özyinelemelidir (recursive). Özyineleme olmadan da yazılabilir, ama çok zahmetli olur.

> **Hocanın vurgusu:** Özyinelemeli bir algoritma yazarken ilk bilmeniz gereken şey durma noktasıdır. Ağaçta durma noktası, bakılan düğümün `NULL` olmasıdır: Oraya gelindiyse gidecek yer kalmamıştır. Düğüm `NULL` ise durulur, değilse iş yapılır.

Preorder'ın mantığı şöyledir: Düğüm `NULL` değilse değeri yazdırılır, sonra aynı fonksiyon sol çocuk için, sol tamamen bittikten sonra da sağ çocuk için çağrılır. `treeNode` `NULL` olduğunda sol bitmiş demektir.

```
void preOrder(NODE *treeNode)
{
    if (treeNode != NULL)
    {
        printf("%d\n", treeNode->value);
        preOrder(treeNode->left);
        preOrder(treeNode->right);
    }
}
```

1.3 Postorder Dolaşma

Hoca postorder'ı daha uzun bir ifadeyle gösterdi: $a * b + c$. Ağaçta kökte `+`, solda `*` (çocukları `a` ve `b`), sağda `c` vardır.

Postorder'da önce sol alt ağaç bitirilir. Sol taraf bir alt ağaç olduğuna göre, onu bitirmek için önce onun solu (`a`), sonra sağı (`b`), sonra kökü (`*`) yazılır. Sonra sağ alt ağaca geçilir. Sağ alt ağaç tek başına `c`'den oluşur. En sonda kök `+` gelir. Sonuç:

```
a b * c +
```

Kod aynı mantıkla yazılır. Durma noktası yine `NULL` olmaktır. Yalnızca iş yapma sırası değişir: sol, sağ, kök. Hocanın dediği gibi burada en kolay çözüm özyinelemeli çözümdür. Klasik (döngülü) çözüm de yazılabilir, ama çok uzun ve zahmetli olur.

```
void postOrder(NODE *treeNode)
{
    if (treeNode != NULL)
    {
        postOrder(treeNode->left);
        postOrder(treeNode->right);
        printf("%d\n", treeNode->value);
    }
}
```

Düğüm `NULL` değilse önce sol çağrılır, sol bitince sağ çağrılır. Bakılacak yer kalmadığında, yani çağrılar geri döndüğünde, değer yazılır. Inorder kodunu hoca öğrencilere bıraktı.

2. Neden İkili Arama Ağacı?

2.1 Sıralı Bağlı Listenin Sorunu

Hoca konuya bir soruyla girdi: Küçükten büyüğe sıralı n elemanlı bir bağlı listede (linked list) aranan eleman en kötü durumda kaç karmaşıklıkla bulunur? Cevap n'dir. Aynı elemanlar bir dizide olsaydı en kötü durumda $\log n$ ile bulunurdu.

Bilgilerin sıralı olması erişimi kolaylaştırır, çünkü ortadan ikiye bölüp işe yaramayan yarıya hiç bakmayabiliriz. Dizide ortadaki elemana (hocanın çiziminde 9) bir seferde erişiriz, çünkü istediğimiz göze doğrudan erişme imkanımız vardır. Bağlı listede ise 9'a erişmek için mecburen 3 ve 5'in üzerinden geçmek gerekir. Bağlı liste, istenen elemana doğrudan erişme imkanı vermez. Sırayla ilerleyince çözüm n karmaşıklıkta kalır ve elemanların sıralı olmasının bir anlamı kalmaz.

2.2 Bağlı Yapı ile Hızlı Aramayı Birleştirmek

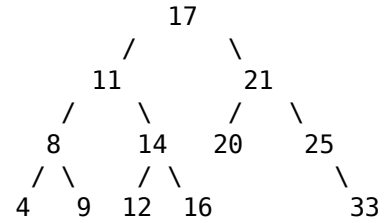
Bazen bilgileri bağlı yapıda saklamak gerekir. Yapı sürekli hareketlidir: Yeni elemanlar gelir, eskiler çıkar. Eleman sayısı önce 3 olur, sonra 30'a çıkar, sonra 7'ye düşer. Bu kadar oynak bir durumda statik bir dizi uzunluğu tanımlamak istemeyiz. Yine de elemanlar sıralı erişilebilirdir ve arama yaparken hızlı bir sistem isteriz.

O zaman elemanlar bir ikili arama ağacı (binary search tree, BST) mantığıyla saklanır. Hoca bunun BST'nin tek kullanım amacı olmadığını, ama önemli amaçlarından biri olduğunu söyledi.

BST özelliği. Kökte x değeri varsa sol alt ağaçtaki bütün değerler x'ten küçük, sağ alt ağaçtaki bütün değerler x'ten büyük veya eşittir. Aynı kural her alt ağaç için de geçerlidir.

2.3 Örnek: 14'ü Aramak

Hocanın çizdiği ağaç şöyledir:



Sol alt ağaçtaki bütün değerler 17'den küçük, sağ alt ağaçtakiler 17'den büyüktür. 11'in alt ağacında da solda 11'den küçükler, sağda 11'den büyükler vardır.

$x = 14$ aranırken iş bir böl ve yönet (divide and conquer) işine dönüşür:

- $14 < 17$ olduğu için sola gidilir. Sağ tarafın tamamı bir seferde elenir.
- $14 > 11$ olduğu için sağa gidilir. 11'in solu da elenir.
- 14 bulunur.

Böylece iki şey birden sağlanır: Bağlı yapıda bir ağaç vardır ve her seferinde yarısı elendiği için arama $\log n$ karmaşıklıkta yapılır. Ağaçta olmayan bir eleman için de "yok" cevabı aynı karmaşıklıkla verilir.

2.4 Düğümün Bellekteki Görünümü

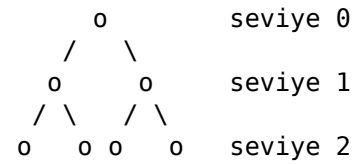
Bağlı yapıda her düğümde bir değer ve iki işaretçi (pointer) vardır. Sol işaretçi soldaki alt ağacın, sağ işaretçi sağdaki alt ağacın adresini tutar. 17'nin `left`'i 11'i, `right`'ı 21'i gösterir. 11'in `left`'i 8'i, `right`'ı 14'ü gösterir ve böyle devam eder. Hocanın örneğinde soldaki düğüm 1000, sağdaki düğüm 1200 adresindedir. Düğüm bu adresleri tutarak bir sonraki sol ve sağ düğümün nerede olduğunu bilir.

Soru: Yeni bir eleman geldiğinde nasıl olacağını gösterecek misiniz?

Cevap: Evet. Önce ağacın genel yapısı anlaşılmalıydı. Eleman ekleme, silme ve arama ayrı ayrı gösterilecek.

3. Ağacın Yüksekliği ve Denge

Karmaşıklık konuşabilmek için önce yükseklik (height) kavramı gerekir. Hoca üç seviyeli, tam dolu bir ağaç çizdi ve yüksekliğini sordu. Bir öğrenci 3 dedi. Doğrusu 2'dir, çünkü kök sıfırıncı seviyedir: Şurası 0., şurası 1., şurası 2. seviye.



Bu ağaçta en fazla 4 yaprak ve 7 düğüm olabilir. Genel sonuçlar şöyledir. Hoca burada genel durumu yeşille, bu ağaca özel sayıları siyahla yazdı.

Yüksekliği h olan ikili ağaç	Genel	$h = 2$ için
En fazla yaprak sayısı	2^h	4
En fazla düğüm sayısı	$2^{h+1} - 1$	7
En az yaprak sayısı	1	1
En az düğüm sayısı	$h + 1$	3

Soru: En az yaprak sayısı h midir?

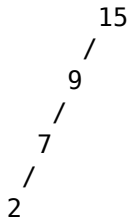
Cevap: Hayır. Herhangi bir ağaçta en az yaprak sayısı 1'dir. Öncekiler bu ağaca özel sayılardı, burada genel durum soruluyor.

En az düğüm sayısı $h + 1$'dir: h seviye varsa her seviyede en az bir düğüm olmak zorundadır. Kök sıfırıncı seviye olduğu için tek düğümlü ağaçta $h = 0$ ve düğüm sayısı 1'dir.

n düğümlü bir ağacın yüksekliği, **ağaç dengeli (balanced) ise** $\log_2 n$'dir.

> **Hocanın vurgusu:** Bu sonuç ancak ağaç dengeliyse geçerlidir. Hoca "bu çok önemli" dedi.
>
> **Ders sonrası not:** Hoca yüksekliği "$\log_2 n$" olarak söyledi; tam değer $\lfloor \log_2 n \rfloor$'dir. Yukarıdaki ağaçta $n = 7$, $\lfloor \log_2 7 \rfloor = 2$'dir. Karmaşıklık için ikisi arasında fark yoktur.

Her seviyede tek düğüm bulunan bir ağaçta ise durum farklıdır. Örneğin 15, 9, 7, 2 sırayla hep sola eklenmişse 4 düğüm vardır, ama yükseklik 3'tür. Bu dengesiz (unbalanced) bir ağaçtır. Kök 15 olunca yeni gelen elemanlar çaresiz sona doğru yığılmıştır.

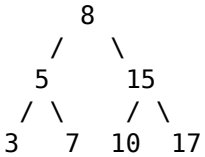


Normal bir ikili ağaçta bu konuda yapılabilecek bir şey yoktur, ağaç sola ya da sağa kaymış olabilir. Ama ikili arama ağacında esas hedef arama yapmaktır. Dengeli ağaçta aranan eleman $O(\log n)$ ile bulunur ya da yoksa "yok" denir. Dengesiz ağaçta ise en kötü durumda $O(n)$ gerekir.

> **Hocanın vurgusu:** Bir ikili arama ağacı oluştururken birinci prensip ağacın dengesini korumaya çalışmaktır. Ağaç normal kullanımda dengesini kaybedebilir. Belli aralıklarla ağacı dengesini toparlayacak şekilde yeniden düzenlemek doğru bir yaklaşımdır.

4. İkili Arama Ağacına Ekleme

Örnek ağaç:



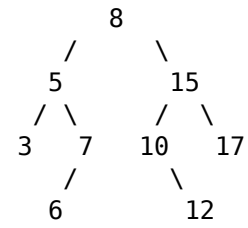
6'yı eklemek. Önce yeni değerin nerede olması gerektiği bulunur. $6 < 8$ iken 6 sağ tarafa konamaz, sol tarafta bir yerde durmalıdır.

- $6 < 8$: sola.
- $6 > 5$: sağa.
- $6 < 7$: sola gidilecek, ama 7'nin çocuğu yok. 6, 7'nin sol çocuğu olur.

  Hocanın vurgusu: Eklemede yeni düğüm her zaman yaprak olarak yerleştirilir. Araya yerleştirmek, örneğin 5'in altına 6'yı koyup 7'yi aşağı kaydırmak, zahmetli bir iştir ve doğru değildir.

12'yi eklemek.

- $12 > 8$: sağa.
- $12 < 15$: sola.
- $12 > 10$: sağa gidilecek, ama 10'un çocuğu yok. 12, 10'un sağ çocuğu olur.



Karmaşıklık. En iyi durum (best case) ağacın tamamen dengeli olduğu durumdur. Aslında o kadar büyük bir denge beklememek gerekir. Ortalama bir ağaç tam dengeli de değildir, çok kötü de. Elemanlar sürekli tek tarafa doğru yığılmışsa ağaç çok kötü bir durumdadır. Hocanın deyişiyle bu yapının ikili arama ağacıyla bir ilgisi kalmamıştır: Bağlı listedeki durum neyse o olmuştur, sırayla ilerlemek zorunda kalırsınız.

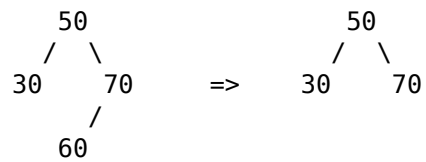
- **En iyi durumda** arama da ekleme de $O(\log n)$'dir.
- **En kötü durumda** (worst case) ikisi de $O(n)$'dir. Hoca tek tarafa yığılmış ağaca 18 eklemeyi örnek verdi: Ağacın sonuna kadar gitmek gerekir. Dengeli sayılabilecek ağaca eklemede ise böyle bir sıkıntı yoktur.

5. İkili Arama Ağacından Silme

Silmede üç durum vardır.

5.1 Durum 1: Silinecek Düğüm Yapraksa

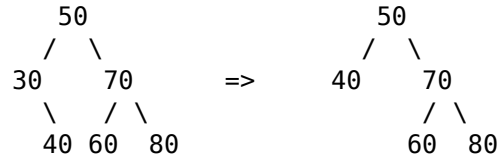
En kolay durum budur. Ağaç şöyle olsun ve 60 silinsin:



Yaprakların işaretçileri zaten `NULL` gösterir. 70'in sol işaretçisi artık 60'ı değil `NULL`'ı gösterir. Bu kadar.

5.2 Durum 2: Silinecek Düğümün Tek Çocuğu Varsa

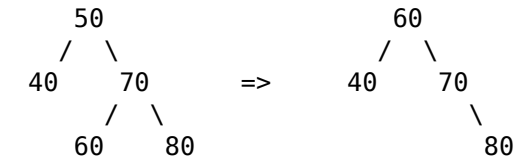
Şimdi 30'un tek çocuğu 40 olsun ve 30 silinsin:



40'ın 50'nin altına gelmesi gerekir. Yapılan iş kolaydır: 50 artık 30'u değil 40'ı gösterir ve 30'un yeri `free` ile serbest bırakılır.

5.3 Durum 3: Silinecek Düğümün İki Çocuğu Varsa

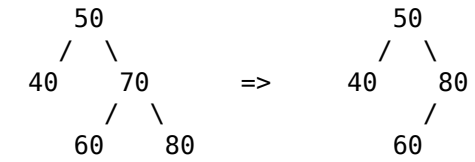
Bu sıkıntılı durumdur. Hoca önce 70'i silmeyi göstermeye başladı, sonra örneğin 50 üzerinden yapılması gerektiğini söyledi. Ağaç şöyle olsun ve kök 50 silinsin:



50 silineceğine göre oraya bir düğüm getirmek gerekir. Bu düğüm ya sol alt ağacın en büyük düğümü ya da sağ alt ağacın en küçük düğümü olmalıdır ki ağacın özelliği korunsun. Genelde sağ alt ağacın en küçüğü seçilir. Hoca bir ara "sağ alt ağacın kökündeki eleman" dedi, hemen özür dileyip "sağ alt ağacın en küçük elemanı" diye düzeltti.

Sağ alt ağacın en küçüğü 60'tır. 50 silinir, 60 köke alınır. Yapılan iş kısaca şudur: Sağ taraftaki en küçük eleman bulunur ve silinecek elemanın yerine getirilir.

Aynı ağaçta 70 silinmek istense, sağ tarafta yalnızca 80 olduğu için 80, 70'in yerine alınır.



Soru: 50'yi silerken yalnızca değeri mi atıyoruz, linklere dokunmuyor muyuz?

Cevap: Linklerle de oynuyorsunuz. Örneğin 70 silinirken 50 artık 70'i değil 80'i gösterir. 50'yi silme örneğinde kök 60 olur: Ağacın başını tutan `head` önce 50'yi gösteriyordu, artık 60'ı gösterir.

Soru: 60'tan sonrasını, 70'teki linkleri de düzenlememiz gerekiyor mu?

Cevap: Hoca hayır dedi: Sağın en küçük elemanı yapraktır, çocuğu yoktur. Onu yukarı çıkardığınızda o tarafta kendisinden büyük elemanlar kalır, bir şey bozulmaz.

> **Ders sonrası not:** Hoca, sağ alt ağacın en küçük elemanının "hep yaprak" olacağını söyledi; bu her zaman doğru değildir. O elemanın sol çocuğu olamaz, çünkü olsaydı en küçük o olmazdı, ama sağ çocuğu olabilir. Bu durumda sağ çocuk, taşınan düğümün eski ebeveynine bağlanmalıdır.
>
> **Ders sonrası not:** Hoca "60'tan sonrasını düzeltmezsin" dedi; düğümler işaretçilerle taşınıyorsa linkler de güncellenmelidir. 50 silinirken 60'ın `left`'i 40'ı, `right`'ı 70'i göstermeli, 70'in `left`'i `NULL` olmalıdır. 70 silinirken de 80'in `left`'i 60'ı göstermelidir, yoksa 60 ağaçtan kopar. Öğrencinin sorduğu yol da geçerlidir: Yalnız değer kopyalanır (`50` yerine `60` yazılır) ve sağ alt ağaçtaki eski düğüm durum 1 ya da 2 ile silinir. Bu yolla iki örneğin sonucu da yukarıdaki ağaçlarla aynı çıkar (python ile denendi).
>
> **Ders sonrası not:** Hoca "ağaç dengelenmiş olarak bitirilir" dedi ve sol alt ağacın en büyüğünü bulmanın "daha zahmetli" olduğunu söyledi; silme işlemi ağacı dengelemez, yalnızca BST özelliğini korur. İki seçim de simetriktir: Sağın en küçüğü için hep sola, solun en büyüğü için hep sağa gidilir.

Hoca silme fonksiyonunun kodunu derste yazmadı. Biraz kurcalayıp, internetteki kodlara bakmadan, yavaş yavaş kendi başınıza yazmanızı önerdi.

6. İkili Ağaç Neden Dizide Değil de Bağlı Yapıda Saklanır?

Hoca aradan sonra, geçen derste konuşulan bir noktayı tekrar hatırlattı. İkili ağaç dizide saklanırken kök 0. gözdedir ve i. gözdeki düğümün sol çocuğu $2i+1$., sağ çocuğu $2i+2$. gözdedir. Yani 0. elemanın çocukları 1. ve 2., 1. elemanın çocukları 3. ve 4., 2. elemanın çocukları 5. ve 6. gözdedir.

Örneğin yalnızca 0., 1., 2. ve 6. gözlerde eleman olsun. 3., 4. ve 5. gözler boş bırakılır. 6. gözdeki düğümün sağ çocuğu $2 \cdot 6 + 2 = 14$. gözdedir. Burada da eleman varsa aradaki bütün gözler boştur. Onun sol çocuğu da $2 \cdot 14 + 1 = 29$. gözdedir. Çok ciddi boşluklar oluşur. Dizi kullanmanın sıkıntısı budur.

> **Hocanın vurgusu:** Tasarımınızda çok fazla boşluk olmayacağından eminseniz dizi kullanmak bağlı listeden daha verimlidir, çünkü istediğiniz göze bir seferde erişirsiniz. Yapı çok hareketliyse ve aralarda boşluklar oluyorsa ideal olan bağlı yapıdır: Boş yer harcanmaz, ağaç dengesiz de olsa sıkıntı yaşamadan çözüm olur.

7. İkili Arama Ağacının Kodları

7.1 Düğüm Yapısı ve `newNode`

Bir düğümün bir değeri, bir sol işaretçisi, bir sağ işaretçisi vardır. Değer birden fazla bilgi içeren bir `struct` da olabilirdi: ad, soyad, numara, adres gibi. Hoca kolaylık için tek bir `int` kullandı. Alt ağaçlar da aynı özellikte olduğu için işaretçiler `struct node` türündedir.

Bağlı bir yapıda her yeni düğüm için yer açmak gerekir. `newNode` bunu yapar. Bir geçici işaretçiyle `malloc` üzerinden yer alınır, değer yazılır, sol ve sağ `NULL` yapılır. Bu sırada düğüm "havada durur", kimseyi göstermez, kimse de onu göstermez. Hoca kodu yazarken `return tmp` satırını unuttuğunu sonra fark edip ekledi: Tutulan yerin adresi çağırana döndürülmelidir.

```
typedef struct node
{
    int value;
    struct node *left;
    struct node *right;
} NODE;

NODE *newNode(int x)
{
    NODE *tmp;
    tmp = (NODE *)malloc(sizeof(NODE));
    /* derste atlandı: burada tmp == NULL kontrolü yapılmalıdır */
    tmp->value = x;
    tmp->left = NULL;
    tmp->right = NULL;
    return tmp;
}
```

> **Hocanın vurgusu:** Derste kolaylık olsun diye yer alındığı varsayıldı, ama `malloc` sonrasında yerin gerçekten alınıp alınmadığı her zaman kontrol edilmelidir. Bir problem olduğunda program sırf bu yüzden çalışmayabilir.

7.2 `insert`

Eklenecek yeri kökten aşağı doğru bulabilmek için fonksiyona ağacın kökü ve x değeri verilir. İlk kontrol, kökün `NULL` olup olmadığıdır. `root` `NULL` ise ortada henüz ağaç yoktur. Gelen ilk düğüm için `newNode` çağrılır ve dönen adres geri verilir. Ağaç boş değilse x kökten küçükse sola, büyük veya eşitse sağa gidilir.

```
NODE *insert(NODE *root, int x)
{
    if (root == NULL)            /* ağaç boş ise */
        return newNode(x);
    if (x < root->value)
        root->left = insert(root->left, x);
    else
        root->right = insert(root->right, x);
    return root;
}
```

Soru: İlk `if`'te `root`'u yeni düğüme atamamız gerekmiyor muydu?

Cevap: Onu dışarıda yapacağız, çünkü fonksiyon adresi `return` ile döndürüyor. Hoca bunun önemli ve güzel bir soru olduğunu söyledi.

9'u eklemenin adım adım izi. Hocanın belleği şöyle varsayalım: Kök düğüm 800 adresinde ve değeri 15. Sol çocuğu 1000 adresinde ve değeri 11. Sağ çocuğu 1200 adresinde ve değeri 17. 11 ve 17'nin çocukları `NULL`. `root` değişkeni 800'ü gösterir, böylece ağaç "uzay boşluğunda" kaybolmaz.

- `insert(800, 9)`: `root` `NULL` değil. $9 < 15$ olduğu için `insert(root->left, 9)`, yani `insert(1000, 9)` çağrılır.
- `insert(1000, 9)`: `root` `NULL` değil. $9 < 11$ olduğu için `insert(NULL, 9)` çağrılır, çünkü 11'in altında başka sayı yoktur.
- `insert(NULL, 9)`: `root` `NULL`. `newNode(9)` çağrılır. Örneğin 1100 adresinde yer alınır, 9 yazılır ve 1100 döndürülür.
- 1100, çağrı `root->left` için yapıldığı için 11'in `left` alanına atanır. Sağ için çağrılsaydı `right` alanına atanacaktı.
- Çağrılar `return root` ile geri döner. Sonuçta 9 ağaca bağlanmıştır.

Yani önce yer bulunur, sonra `malloc` ile yer alınır, alınan adres de ilgili işaretçiye atanır.

7.3 Ana Programda Kullanım

Başlangıçta `root` `NULL`'dır. Hoca önce şu program parçasını gösterdi. İlk çağrıda 15 eklenir ve tutulan adres `root`'a döner. Diğer düğümlerin adreslerini saklamaya gerek yoktur, onlar yapının içinde oluşur.

```
NODE *root = NULL;

root = insert(root, 15);
insert(root, 17);
insert(root, 11);
insert(root, 9);
```

Soru: `root`'un adresini zaten gönderiyoruz. `return root` yazmasak da olur mu?

Cevap: Özyinelemeli yapıldığı ve `return newNode` kullanıldığı için `return root` gereklidir. Hoca ilk başta emin olmadığını söyledi, sonra ana programı daha anlaşılır olması için şöyle değiştirdi: İlk düğüm 15 doğrudan kök olarak tanımlanır, sonrakiler `insert` ile eklenir. İki yazım da doğrudur.

```
root = newNode(15);
insert(root, 17);
insert(root, 11);
insert(root, 9);
```

> **Ders sonrası not:** Hoca bu soruda "muhtemelen" ve "çok emin değilim" diye tereddüt etti; durum nettir. `insert` içindeki `root->left = insert(...)` atamaları dönüş değerine dayandığı için `return root` şarttır. `root` işaretçisi fonksiyona değer olarak (by value) geçer. Boş ağaçta `insert(root, 15);` yazıp dönen değer atanmazsa ana programdaki `root` `NULL` kalır (derlenip denendi). Ağaç boş değilken kök değişmediği için ana programda dönüş değerini atamamak sorun çıkarmaz.

Hoca sonra şunu sordu: Bu ağacı küçükten büyüğe sıralı yazdırmak için ne yapmalıyız? Bir öğrenci inorder dolaşma dedi ve hoca doğruladı.

> **Hocanın vurgusu:** İkili arama ağacını inorder dolaşırsanız elemanları küçükten büyüğe sıralı görürsünüz.

7.4 `search`

Aramak için ağacın kökü ve aranan x verilir. İlk durum, elemanın hemen kökte olması ya da ağacın boş olmasıdır. İki durumda da `root` döndürülür. `NULL` kontrolünü alan taraf yapabilir,

isterseniz fonksiyonun içinde de yapabilirsiniz. Sonra x kökten küçükse sol alt ağaca, değilse sağ alt ağaca gidilir.

```
NODE *search(NODE *root, int x)
{
    if (root == NULL || root->value == x)
        return root;
    else if (x < root->value)
        return search(root->left, x);
    else
        return search(root->right, x);
}
```

> **Hocanın vurgusu:** Hoca mecbur kalmadıkça özyinelemeli yazmayı iyi bulmuyor. Fonksiyon çağırmak da sistem için bir yüktür. Özyinelemeli çözümün daha etkili olduğu bir problem yoksa gereksiz yere kullanmayın. Ağaçta ise solda alt ağaç varsa orada, sağda varsa orada ilerleme mantığı olduğu için özyinelemeli çözüm çok daha kolay olabilir.

Hoca arama fonksiyonunun düz (özyinelemesiz) de yazılabileceğini söyledi: Küçükse sola, büyükse sağa git. Ekleme biraz daha uzun olur, yine de düz yazılabilir.

Soru: Ağaçlar bir yandan yığın (stack) da kullanıyor diyebilir miyiz? Çünkü tamamen özyineleme kullanıyoruz.

Cevap: Özyinelemenin yığın kullandığı doğrudur, ama ağaç yığın kullanmaz. Soyut veri tipi (abstract data type) olarak ağaç ya bağlı listeyle ya da diziyle gerçekleştirilir. Ekleme, silme ve arama özyineleme olmadan da yazılır. Hocanın böyle hazırladığı bir programı da var, çok uzun olduğu için derste yazmadı.

> **Ek bilgi (derste anlatılmadı):** Yukarıdaki parçalar birleştirilip `gcc -Wall` ile derlendi ve hiç uyarı vermedi. Inorder fonksiyonu derste yazılmadığı için, anlatılan "sol, kök, sağ" sırasıyla buraya eklendi. `main` içindeki yazdırma ve `search` satırları da doğrulama için eklendi; 14 derste aranan değerdir ve bu ağaçta yoktur.

```
#include <stdio.h>
#include <stdlib.h>

/* NODE, newNode, insert, search ve postOrder burada, yukarıdaki gibi */

void inOrder(NODE *treeNode)
{
    if (treeNode != NULL)
    {
        inOrder(treeNode->left);
        printf("%d\n", treeNode->value);
        inOrder(treeNode->right);
    }
}

int main(void)
{
    NODE *root = NULL;

    root = newNode(15);
    insert(root, 17);
    insert(root, 11);
    insert(root, 9);

    printf("inorder:\n");
    inOrder(root);                  /* 9 11 15 17 */
    printf("postorder:\n");
    postOrder(root);                /* 9 11 17 15 */
```

```
    printf("9: %s\n", search(root, 9) != NULL ? "bulundu" : "yok");
    printf("14: %s\n", search(root, 14) != NULL ? "bulundu" : "yok");
    return 0;
}
```

8. Ara Bilgi: Bellekteki Heap Bölgesi

Hoca heap ağacına geçmeden önce "heap" kelimesi karışmasın diye kısa bir açıklama yaptı ve bunun konuyla hiç ilgisi olmadığını özellikle belirtti. Programlama derslerinden akılda kalması gereken heap bir bellek bölgesidir.

Bir fonksiyon içinde yerel bir `A` dizisi tanımlanıp `return A` yazıldığını düşünelim. `A` yerel değişken olduğu için fonksiyondan çıkınca orası serbest bırakılır. Başka biri orayı kullanırsa `A` adresinde alakasız değerler görülür. Ama `A` bir işaretçi olarak tanımlanıp `malloc` ile yer alınır ve `return A` yazılırsa içindeki değerler kaybolmaz. Çünkü dinamik olarak alınan bu adresler heap bölgesindedir ve program çalıştığı sürece korunur.

> **Ders sonrası not:** Hoca, yerel dizinin adresinin `return` ile "döneceğini", yalnız içindeki değerlere güvenilemeyeceğini söyledi; yerel değişkenin adresini fonksiyon dışında kullanmak tanımsız davranıştır (undefined behavior). `gcc -Wall` bu satırda "function returns address of local variable" uyarısı verdi. Burada kullanılan gcc 15.2 ile derlenen programda fonksiyon adres yerine `NULL` döndürdü. Yani adresin döneceğine de güvenilemez.
>
> **Ders sonrası not:** Hoca, global değişkenlerin de heap bölgesinde saklandığını ve bu bölgenin CPU tarafından düzenlendiğini söyledi; doğrusu farklıdır. Global değişkenler veri bölümündedir: Bu makinede derlenen programda `nm` çıktısı ilk değerli global için `D` (data), ilk değersiz global için `B` (BSS) gösterdi. Heap bölgesini CPU değil, `malloc` ve `free` gibi bellek ayırma fonksiyonları işletim sistemiyle birlikte yönetir. `malloc` ile alınan yer de "program boyunca" değil, `free` edilene ya da program bitene kadar geçerlidir.

Heap ağacındaki "heap" ile bu bellek bölgesinin hiçbir ilgisi yoktur.

9. Heap Ağacının Üç Özelliği

Hoca öncelikli kuyruk (priority queue) yapısının çok kullanıldığını söyledi ve ayrık matematikten bilinen Huffman ağacını hatırlattı. Huffman ağacı aslında öncelikli kuyruk yapısı kullanılarak tasarlanan bir yapıdır. Öncelikli kuyruk ve Huffman bu derste değil, sınavdan sonra anlatılacak. Bu derste heap ağacı (heap tree) anlatıldı.

Heap ağacı üç özelliği birden taşır: dengelidir, sola dayalıdır ve heap özelliğine sahiptir. Hocanın ilk verdiği tanımda (max heap) düğümler ebeveynlerinden büyük değer alamaz, ama ikili arama ağacındaki gibi "küçükler bir tarafta, büyükler bir tarafta" kuralı yoktur.

9.1 Dengeli (Balanced) Olma

Az önce genel bir denge kavramı görülmüştü. Gerçek anlamda dengeli ikili ağaç için şu gerekir: Yaprakların olduğu en alt katman n. katman ise, 0. derinlikten $n-2$. derinliğe kadar **bütün düğümlerin iki çocuğu** vardır. $n-1$. katmanda bir çocuğu olan da olabilir, hiç çocuğu olmayan da. Bunlar sorun değildir. $n-2$. katmanda çocuğu eksik bir düğüm varsa ağaç dengeli değildir.

```
Dengeli                          Dengeli değil

        o        0                       o        0
      /   \                            /   \
     o     o     n-2                  o     o     n-2
    / \   / \                        / \     \
   o   o o   o   n-1                o   o     o   n-1
  / \                              /
 o   o           n                o               n
```

9.2 Sola Dayalı (Left Justified) Olma

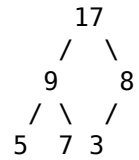
Sola dayalılıkta yapraklara bakılır: Hepsi sola dayalı olmalıdır. Aynı seviyede olmaları gerekmez. En alt seviyede soldan sağa doğru arada boşluk varsa ağaç sola dayalı değildir. Hoca bunun nedeninin birazdan görüleceğini söyledi.

9.3 Heap Özelliği

İki tür heap vardır, ihtiyaca göre biri seçilir:

- **Max heap:** Ebeveyn her zaman çocuklarından büyüktür. Kökte en büyük değer bulunur.
- **Min heap:** Ebeveyn her zaman çocuklarından küçüktür. Kökte en küçük değer bulunur.

Hocanın max heap örneği şöyledir:



Burada bir yerde büyük solda küçük sağda, bir yerde tersidir. Bunun bir kuralı yoktur. Bu ağaç ikili arama ağacı değildir. Önemli olan ebeveynin çocuklarından büyük olmasıdır. Sol ve sağ alt ağaçların birbiriyle bir ilgisi yoktur.

Hoca min heap için önce ebeveyn 11, çocuklar 7 ve 6 dedi. Bir öğrenci bunun ters olduğunu söyleyince çocukları 17 ve 16 olarak düzeltti: Min heap'te ebeveyn çocuklarından küçüktür.

Bir düğümün değeri 12 olsun, çocuklarından biri 12'den küçük, biri 12'den büyük olsun. Bu ağaç ne max ne de min heap olur. Ya hepsi küçük ya hepsi büyük olmalıdır.

Soru: Bunun ne işe yaradığını anlamaya çalışıyoruz, anlatacak mısınız?

Cevap: Evet. Önce heap mantığının anlaşılması gerekir ki kullanım yerleri söylendiğinde akılda rahat otursun.

9.4 Neden Dengeli ve Sola Dayalı?

> **Hocanın vurgusu:** Ağaç hem dengeli hem sola dayalı olduğu için arada boş göz kalmaz. Kural bunu gerektirir. Bu yüzden heap ağacı bağlı yapı yerine normal bir dizide rahatça saklanabilir.

İkili ağaçlarda karşılaşılan aralarda boşluk olma problemi heap ağacında yoktur. Uygulamaya göre bu değişebilir. Hoca öncelikli kuyruklarda başka örnekler de vereceğini söyledi.

10. Heap Ağacına Eleman Ekleme

Boş bir ağaçtan başlanır ve bir min heap kurulur. Her yeni eleman, ağacın en son gözüne (ilk boş göze) konur. Sonra uygun hale getirilir. Elemanın yerini bulma, ağacı yeniden heap özelliğine kavuşturma işlemine hoca heapify dedi. Yukarı doğru taşımanın özel adı sift up, aşağı doğru taşımanınki sift down'dur.

Gelen elemanlar sırayla 8, 4, 9, 5, 1 olsun.

```
8 geldi: [8]

4 geldi: [8, 4]
  4 < 8, yer değiştir     -> [4, 8]

9 geldi: [4, 8, 9]
  9 > 4, dokunma          -> [4, 8, 9]

5 geldi: [4, 8, 9, 5]     (2. gözden sonraki ilk boş yer 3. gözdür)
  5 < 8, yer değiştir     -> [4, 5, 9, 8]
  5 > 4, dur
```

```
1 geldi: [4, 5, 9, 8, 1]  (ilk boş yer 4. gözdür)
  1 < 5, yer değiştir     -> [4, 1, 9, 8, 5]
  1 < 4, yer değiştir     -> [1, 4, 9, 8, 5]
```

Son ağaç şöyledir:

```
      1
     / \
    4   9
   / \
  8   5
```

Hoca bu sonucu önce "1, 4, 8, 5, 9" diye okudu, hemen ardından dizi olarak "1, 4, 9, 8, 5" diye doğru haliyle tekrar etti.

Ekleme sırasında yalnızca yeni elemanın yolu izlenir. 5 eklenirken sağ alt ağaca hiç ulaşılmaz: 5 yalnızca 8 ve 4 ile karşılaştırılır, 9 ile karşılaştırılmaz.

11. Heap Ağacından Kökü Silme

Soru: Bu ağaçta köke bakınca ne görüyorsunuz?

Cevap: Ya en küçük ya en büyük eleman.

Heap ağacının genel kullanımında yapılan işlem kökle ilgilidir, çünkü en küçük ya da en büyük elemana erişmek isteriz. Öncelikli kuyrukta da bu görülecek. Aradaki bir elemana genelde ihtiyaç duyulmaz. Bu yüzden silme işlemi kök üzerinden yapılır.

Kökü silmek için kök ile en son eleman yer değiştirir. İhtiyaca göre çıkan eleman orada kalabilir ya da kaldırılıp götürülebilir, o göz artık kullanılmaz. Sonra ağaç yeniden heap haline getirilir. Bu kez eleman yukarıdan aşağı taşınır.

```
[1, 4, 9, 8, 5]
1 ile 5 yer değiştirir, 1 çıkar      -> [5, 4, 9, 8]
5'in çocukları 4 ve 9: küçüğü 4
4 < 5, yer değiştir                  -> [4, 5, 9, 8]
5'in tek çocuğu 8: 5 < 8, dur
```

Soru: 5'i aşağı indirirken kimle karşılaştırmak gerekir, 4 ile mi?

Cevap: Hayır. Hoca sordu: 9 yerine 3 olsaydı ne yapardınız? O zaman 4 değil 3 yukarı çıkmalıydı. Bu yüzden önce iki çocuk birbiriyle karşılaştırılır, çocukların en küçüğü seçilir, sonra aşağı inen eleman onunla karşılaştırılır.

> **Hocanın vurgusu:** Sift down'da önce iki çocuğun birbirine göre durumuna bakmalısınız. Min heap'te küçük olanı, max heap'te büyük olanı seçip aşağı inen elemanla karşılaştırırsınız. Tek bir çocuğa bakmak yanlış sonuç verir.

Hoca bunu bir varsayımla pekiştirdi: Ağaç `[4, 5, 9, 8, 7]` olsaydı, 5'in çocukları 8 ve 7 olurdu. $8 < 7$ değildir, küçük olan 7'dir. $5 < 7$ olduğu için 5 yerinden kıpırdamaz.

Soru: Bir düğümün kaç çocuğu olduğunu, 1 mi 2 mi, nasıl anlıyoruz?

Cevap: Eleman sayısı n'e bakarak. Örneğimizde $n = 4$ ve gözler 0, 1, 2, 3'tür. 5, 1. gözdedir. Sağ çocuğu $2i + 2 = 4$. gözde olurdu, ama 4. göz yoktur. Yani yalnız sol çocuğu vardır. Ağaç $n = 3$ olsaydı $2i+1 = 3$. göz de olmazdı. O zaman ağacın bittiğini ve 5'in yaprakta durduğunu anlardık.

Genel kural şöyledir: $2i + 1 \geq n$ ise düğümün çocuğu yoktur. $2i + 2 \geq n$ ise (ve $2i + 1 < n$ ise) yalnız sol çocuğu vardır.

12. Heap Ağacının Dizide Saklanması

Hoca heap sort'a geçmeden önce bir max heap'in diziye nasıl yerleştiğini gösterdi: 0. gözde 15, 1. gözde 9, 2. gözde 8, 3. gözde 4 vardır. i. gözdeki düğümün çocukları $2i + 1$. ve $2i + 2$. gözlerdedir.

```
      15
     /  \            indis:  0   1   2   3
    9    8           dizi : [15, 9,  8,  4]
   /
  4
```

13. Heap Sort

Heap ağacının en temel kullanım şekli heap sort algoritmasıdır. Bir diziyi sıralar. Karmaşıklığı $n \log n$'dir.

Önce heap özelliğine sahip bir ağaç gerekir. Hoca önce büyükten küçüğe sıralamayı düşündü, ağacı max heap kurduğu için sonra küçükten büyüğe sıralayacağını söyledi. Kökte en büyük eleman vardır. Küçükten büyüğe sıralarken en büyük eleman en sonda olmalıdır. Bu yüzden kök en sona alınır.

Her adımda yapılan iş şudur:

- Kök ile bakılan bölümün son elemanı yer değiştirir. Kök artık yerine yerleşmiştir, ona bir daha hiç uğranmaz.
- Köke gelen eleman sift down ile aşağı indirilir. Önce iki çocuk karşılaştırılır, büyüğü seçilir, sonra aşağı inen elemanla karşılaştırılır.

13.1 Örnek 1

Başlangıç max heap'i: `[45, 22, 36, 13, 5, 21, 19, 8]`.

```
          45
        /    \
      22      36
     /  \    /  \
   13    5  21   19
  /
 8
```

1. adım. 45 ile 8 yer değiştirir. 45 yerleşti.

8 aşağı inecek. Hoca burada önce yanlışlıkla 8 ile 22'yi karşılaştırmaya başladı, özür dileyip düzeltti: Köke birini almak gerektiği için önce iki çocuk karşılaştırılır. 22 ile 36'dan büyüğü 36'dır, 8 ile 36 yer değiştirir. 8'in yeni çocukları 21 ve 19'dur. Büyüğü 21'dir, 8 ile 21 yer değiştirir. Sol tarafa hiç bakılmadı. Bu sırada bir öğrencinin sorusu üzerine hoca, 13 ile karşılaştırma yapılmayacağını da söyledi.

2. adım. 36 ile 19 yer değiştirir. 19 için 22 ile 21 karşılaştırılır, 22 büyüktür, 19 ile yer değiştirir. Sonra 19'un çocukları 13 ve 5'tir. 13 büyüktür, ama 19 ondan da büyüktür, 19 yerinde kalır.

> **Ders sonrası not:** Hoca bu adımda "22 ile 21'i karşılaştırdım, 22 küçük" dedi; doğrusu 22 büyüktür. Hemen sonraki yer değiştirme ve sonuç doğrudur.

3. adım. 22 ile 8 yer değiştirir. 19 ile 21'den büyüğü 21'dir, 8 ile 21 yer değiştirir. 8'in altında kimse kalmamıştır.

4. adım. 21 ile 5 yer değiştirir. 19 ile 8'den büyüğü 19'dur, 5 ile 19 yer değiştirir. 5'in tek çocuğu 13'tür, 13 büyüktür, 5 ile 13 yer değiştirir.

5. adım. 19 ile 5 yer değiştirir. 13 ile 8'den büyüğü 13'tür, 5 ile 13 yer değiştirir. 5'in altında kimse yoktur.

6. adım. 13 ile 8 yer değiştirir. 8 ile 5 karşılaştırılır, 8 büyüktür, 8'e dokunulmaz.

7. adım. 8 ile 5 yer değiştirir. 5 tek düğüm kaldığı için yapacak bir şey yoktur.

Adım	Yer değiştiren	Sift down sonrası heap kısmı	Yerleşmiş kısım
1	45 ile 8	`36, 22, 21, 13, 5, 8, 19`	`45`
2	36 ile 19	`22, 19, 21, 13, 5, 8`	`36, 45`
3	22 ile 8	`21, 19, 8, 13, 5`	`22, 36, 45`
4	21 ile 5	`19, 13, 8, 5`	`21, 22, 36, 45`
5	19 ile 5	`13, 5, 8`	`19, 21, 22, 36, 45`
6	13 ile 8	`8, 5`	`13, 19, 21, 22, 36, 45`
7	8 ile 5	`5`	`8, 13, 19, 21, 22, 36, 45`

Sonuç sıralı dizidir: `5, 8, 13, 19, 21, 22, 36, 45`.

14. Heap Sort'un Karmaşıklığı

Soru: Buna $n \log n$ demiştiniz, değil mi?

Cevap: Hoca bunu iki parçaya ayırarak açıkladı.

Sift down. Yüksekliği $\log n$ olan ağaçta bir elemanı aşağı taşımak en fazla kaç adım sürer? Bir öğrenci "$\log n - 1$ mi?" dedi. Hoca en kötü durumda en aşağı taşınacağı için maksimumu $\log n$ diye alabileceğimizi söyledi. Bazen bir ya da iki adımda biter, ama en kötü durum budur.

Sift up. Yeni eleman en fazla köke kadar taşınır. O da $\log n$'dir. Eleman yerinde kalırsa hiçbir iş yapılmaz. n elemanlı ağaç dengeli olduğu için yüksekliği $\log n$ olmak zorundadır.

Sıfırdan heap kurma. Her elemanın yerleşmesi en kötü durumda $\log n$ olduğuna göre, n eleman için toplam $n \log n$ olur. Yoktan bir heap ağacı kurmanın karmaşıklığı $n \log n$'dir.

Sıralama döngüsü. Sift down bir eleman için $\log n$ sürer. Bu iş dizideki n eleman için yapılacağı için toplam $n \log n$ olur.

Soru: $n \log^2 n$ mi?

Cevap: Hayır. Burada karesiyle bir işimiz yok.

Toplam şöyledir:

$$O(n \log n) + O(n \log n) = O(n \log n)$$

Yani heap sort, merge sort ile aynı karmaşıklıkta, en hızlı sıralama algoritmalarından biridir.

Soru: Önceki slaytta heapify için $n \log n$ mi dediniz?

Cevap: Heap ağacı kurma (construction) için $n \log n$ dendi. Sift down ve sift up bir eleman için $\log n$'dir.

Soru: Heap ağacı kurma aslında $\log n$ değil mi? Heap sort da $n \log n$ değil mi?

Cevap: Bir değeri yerleştirmek $\log n$'dir, aşağı doğru da yukarı doğru da. Ama n değer olduğu için bütün diziyi sıfırdan heap yapmak $n \log n$'dir. Heap sort da $n \log n$'dir: İki tane $n \log n$'nin toplamı yine $n \log n$ eder.

> **Ders sonrası not:** Hoca ilk başta "heapify işleminin karmaşıklığı n" dedi, slaytta da "Heapify the array ⇒ N" yazıyor; sonra heap kurmanın $n \log n$ olduğunu söyledi. İkisi iki ayrı yönteme karşılık gelir. Elemanları tek tek ekleyerek (sift up) kurmak en kötü durumda $O(n \log n)$'dir. Diziyi son iç düğümden köke doğru sift down ile heap yapmak (Floyd yöntemi) $O(n)$'dir: Bu yöntemde yer değiştirme sayısı en fazla $\sum_{h=1}^{\lfloor \log_2 n \rfloor} \lceil n/2^{h+1} \rceil \, h < 2n$ olur. Hangi yöntem seçilirse seçilsin heap sort'un toplamı $O(n \log n)$'dir.

Ders sonrası not: Hoca sıralama döngüsünde "n eleman için" sift down yapıldığını söyledi; döngü $i = n - 1$'den 1'e kadar döndüğü için tam olarak $n - 1$ kez çalışır, yani Örnek 1'de 7 adım vardır. Karmaşıklık sonucu değişmez.

15. Heap Sort'un Algoritması ve Küçük Bir Örnek

15.1 Algoritma

Hocanın tahtaya yazdığı adımlar şöyledir:

- Diziyi sıfırdan heap özelliğine getir (heapify). Karmaşıklığı $n \log n$'dir.
- $i = n - 1$'den 1'e kadar:
  - 0. eleman ile i. elemanı yer değiştir (exchange).
  - İlk i eleman için sift down yap.

Hoca bunu önce $n = 4$ olan bir ağaçla gösterdi: 0. ile $n - 1$. eleman yer değiştirir, 8 köke gelir, 21 sona yerleşir. Artık 3 eleman kalmıştır, son göz bir daha kullanılmaz. Sift down da bu 3 eleman için yapılır.

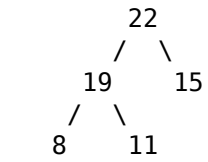
```
void HeapSort(int Heap[], int n)
{
    int i;    /* tahtadaki kodda bildirim yoktu, derlenmesi için eklendi */

    for (i = n - 1; i > 0; i--)
    {
        swap(&Heap[0], &Heap[i]);
        siftDown(Heap, i);    /* şu anda i elemanım kaldı */
    }
}
```

Bir öğrenci bunun karışık geldiğini söyledi ve hocadan i'nin rolünü ağaç üzerinde göstermesini istedi. Hoca da daha küçük bir diziyle tekrar anlattı.

15.2 Örnek 2

$n = 5$, başlangıç max heap'i: `[22, 19, 15, 8, 11]`.



- $i = 4$**:** 0. ile 4. göz yer değiştirir (22 ile 11). 22 yerleşti, artık ona bakılmaz, 4 eleman kaldı. 19 ile 15'ten büyüğü 19'dur. 11 ile 19 karşılaştırılır, 19 büyüktür, yer değiştirirler. Sonuç: `19, 11, 15, 8` ve yerleşmiş `22`.
- $i = 3$**:** 0. ile 3. göz yer değiştirir (19 ile 8). 3 eleman kaldı. 15 ile 11'den büyüğü 15'tir. 8 ile 15 yer değiştirir. Sonuç: `15, 11, 8` ve yerleşmiş `19, 22`. 3. ve 4. gözlere hiç bakılmaz, çünkü i azaldı.
- $i = 2$**:** 0. ile 2. göz yer değiştirir (15 ile 8). Sift down iki eleman için yapılır ($n = 2$ gibi). 8 ile 11'e bakılır, 11 yukarı çıkar. Sonuç: `11, 8` ve yerleşmiş `15, 19, 22`.
- $i = 1$**:** 0. ile 1. göz yer değiştirir (11 ile 8). Bir eleman kaldığı için iş yapılmaz.

i	Yer değiştiren	Sift down sonrası heap kısmı	Yerleşmiş kısım
4	22 ile 11	`19, 11, 15, 8`	`22`
3	19 ile 8	`15, 11, 8`	`19, 22`
2	15 ile 8	`11, 8`	`15, 19, 22`
1	11 ile 8	`8`	`11, 15, 19, 22`

Sonuç: `8, 11, 15, 19, 22`.

Bir elemana artık bakılmamasını sağlayan şey i değişkenidir. i azaldıkça bakılan eleman sayısı da azalır. $i = 0$ için iş yapılmaz: Kökteki eleman zaten en küçük elemandır, herkes yerleşmiştir.

Hoca, "şu ikisini karşılaştır, sonra sonucu kökle karşılaştır, gerekiyorsa yer değiştir" işinin tasarımını zaman kalmadığı için bu derste anlatmadı, sonraya bıraktı.

> **Ek bilgi (derste anlatılmadı):** `siftDown` fonksiyonunun içi derste yazılmadığı için, sözlü anlatılan kuralla burada yazıldı. n kontrolü bölüm 11'deki $2i + 1$ ve $2i + 2$ kuralıdır. Bu kod, `HeapSort` ile birlikte `gcc -Wall` ile uyarısız derlendi ve iki örneğin sonucunu da doğru verdi: `5 8 13 19 21 22 36 45` ve `8 11 15 19 22`.

```c
void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

/* Kökteki elemanı, ilk n elemanlık max heap içinde aşağı indirir. */
void siftDown(int Heap[], int n)
{
    int i = 0, c;

    while (2 * i + 1 < n)              /* sol çocuk yoksa yapraktayız */
    {
        c = 2 * i + 1;
        if (c + 1 < n && Heap[c + 1] > Heap[c])
            c = c + 1;                 /* iki çocuğun büyüğünü seç */
        if (Heap[i] >= Heap[c])
            return;                    /* yerini buldu */
        swap(&Heap[i], &Heap[c]);
        i = c;
    }
}
```

16. Kuyruk Soruları: `isFull`, Dairesel Kuyruk ve Yığınla Kuyruk

Dersin sonunda önceki çalışmalardan kalan kuyruk soruları çözüldü.

16.1 Bağlı Listeyle Kuyrukta `isFull`

Soru: Kuyruk için bağlı liste kullanmıştık. Düz bağlı listede `front` ile `rear` hiçbir zaman birbirine eşit olmaz. O zaman `isFull` nasıl yazılır? Dairesel bağlı liste mi kullanmalıyız?

Cevap: Soruda aslında dairesel dizi düşünülmesi isteniyordu, bağlı liste değil. Çok hareketli bir yapınız yoksa kuyrukta bağlı liste kullanmayın, dairesel kuyruk (circular queue) kullanın.

Dairesel yapmanın nedeni şudur: Kuyruğun başındaki 3 çıktığında herkesi bir öne kaydırmak n karmaşıklıktadır ve zaman alıcıdır. Onun yerine 3 çıkınca kuyruğun başı 5 olur, yeni gelen eleman da 3'ün boşalttığı göze yerleşebilir. Sonra 5 de gidip yerine 4 gelirse, `front` 9'u, `rear` 4'ü gösterir ve bu dolu bir kuyruktur. İstenen, bu doluluk durumunun anlaşılmasıydı.

Bağlı listede ise doluluk diye bir şey pek olmaz, çünkü elemanlar ekleyip çıkarırken daha esneksinizdir. Yine de dinamik bellek ayırmanın da bir sınırı vardır. Orada doluluk şöyle kontrol edilebilir: Eklenen eleman sayısı sayılır ve bir sınır aşılırsa hata verilir (hocanın örneği: bir megabayt yer varsa ondan fazlası eklenince) ya da `malloc` yer ayıramazsa hata verilir.

> **Ek bilgi (derste anlatılmadı):** Dairesel dizide doluluk, `rear`'ın bir sonraki gözünün `front` olmasıyla anlaşılır: `(rear + 1) % N == front`. Yukarıdaki durum da budur, çünkü 4'ten sonraki göz 9'un gözüdür. Aynı eşitlik kuyruk tamamen boşaldığında da oluşabilir. Bu yüzden ayrıca bir eleman sayacı tutulabilir ya da bir göz boş bırakılabilir.

16.2 Dairesel Kuyrukta Yeni Elemanın Yeri

Soru şuydu: Bir kuyruk için N elemanlı dizi kullanılıyor, dairesel olarak gerçekleştiriliyor, kuyruğun son elemanının bulunduğu göz `rear` değişkeninde. Yeni gelen eleman hangi göze yerleşir? Cevap:

```
(rear + 1) % N
```

Hoca farklı durumları gösterdi. `rear` 3. gözdeyse yeni eleman 4. göze gelir. `rear` 1. gözdeyse 2. göze gelir. Bir süre sonra `rear` dizinin son gözüne gelmiş olsun ve dizinin başı da boşalmış olsun. Bu durumda `rear + 1` dizinin dışını gösterir, yeni eleman 0. göze gelmelidir. Bu yüzden N'e göre mod alınır.

Sınıfta verilen iki yanlış cevap şunlardı:

- **`rear + 1`:** Yukarıda gösterilen durumda yanlıştır.
- **`rear->next`:** Soruda dizi deniyor, bağlı liste denmiyor. Dizide `next` diye bir şey yoktur.

Bir de "0. gözdür" diyenler vardı. Bir öğrenci sorunun dizinin sonundaki özel durumu sorup sormadığını sordu. Hoca hayır dedi: `rear` dizinin sonu değil, kuyruğun son elemanının gözüdür. Kuyruğun sonu 1. ya da 2. gözde de olabilir.

16.3 Yığın Kullanarak Kuyruk

Soru şuydu: Yığında `push` ve `pop` dışında yığını ters çeviren bir `reverse` işlemi de var. Yığınla kuyruk gerçekleştirirken `enqueue` ve `dequeue` için ne kullanılır?

- **`enqueue`:** `push`. Gelen elemanlar yığına dümdüz yerleştirilir.
- **`dequeue`:** `reverse`, `pop`, `reverse`.

Hocanın örneğinde yığında alttan üste 3, 5, 9, 11 vardır. Kuyruktan çıkması gereken 3'tür.

```
Başlangıç (alttan üste) : 3, 5, 9, 11
reverse (üstten alta)   : 3, 5, 9, 11    (3 en üste geldi)
pop                     : 3 alındı
reverse (alttan üste)   : 5, 9, 11
```

Hoca sıraları sözlü okudu. Yığının hangi yönden okunduğu ara adım olarak eklendi (ara adım eklendi).

Dersin Toparlaması

Bu derste önce dolaşma yöntemleri hatırlatıldı. Sıralı bağlı listede aramanın neden n karmaşıklıkta kaldığı görüldü. İkili arama ağacının, bağlı yapının esnekliğini koruyarak dengeli durumda $\log n$ arama sağladığı anlatıldı. Yükseklik, denge, ekleme, silmenin üç durumu ve `newNode`, `insert`, `search` kodları işlendi. Silme fonksiyonunu yazmak öğrencilere bırakıldı.

Sonra heap ağacına geçildi. Dengeli ve sola dayalı olduğu için heap ağacı boşluksuz biçimde dizide saklanır. Ekleme sift up, kök silme sift down ile yapılır ve ikisi de $\log n$'dir. Heap sort bu iki işlemin üzerine kurulur: Kök sona alınır, kalan bölüm yeniden düzenlenir ve toplam karmaşıklık $n \log n$ olur. Hocanın sonraya bıraktığı konular, sift down işleminin algoritma tasarımı ile sınavdan sonra anlatılacak öncelikli kuyruk ve Huffman ağacıdır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.